Authentication & Authorization Strategies

Java Backend Interview Reference

Authentication vs Authorization

	Authentication (AuthN)	Authorization (AuthZ)
Question	"Who are you?"	"What can you do?"
Purpose	Verify identity — prove the user is who they claim to be	Enforce permissions — decide what the verified user is allowed to access
When	Before authorization — must happen first	After authentication — requires identity to be established
Failure result	401 Unauthorized ("you are not logged in")	403 Forbidden ("you are logged in but cannot access this")
Examples	Username+password, biometric, MFA, certificate	RBAC roles, ABAC policies, ACL, OAuth2 scopes

Authentication Strategies

Strategy	How It Works	Pros	Cons	Use Case
Username + Password	User sends credentials; server validates against hashed password in DB	Familiar, simple	Phishing, weak passwords, no MFA by default	Traditional web apps
JWT (JSON Web Token)	Server issues signed token on login. Client sends it in Authorization: Bearer header. Server verifies signature — no DB lookup needed.	Stateless, scalable, cross-domain	Cannot invalidate before expiry without blocklist; payload visible (not encrypted)	REST APIs, microservices, mobile
OAuth2 / OIDC	User delegates access to a third party. Resource Server validates access token via introspection or JWT signature.	Industry standard, no password sharing, scoped access	Complex flow, multiple redirects	Third-party login, API authorization
API Key	Static secret in header (X-API-Key) or query param. Server validates against DB or cache.	Simple, stable, machine-to-machine	No expiry by default; leaked key = full access; no user identity	B2B APIs, developer platforms, webhooks
Session Cookie	Server creates session on login, stores in DB/Redis, sends cookie. Client sends cookie on each request.	Revocable instantly, standard browser support	Stateful — server must store sessions; CSRF risk	Traditional web apps, portals
mTLS (Mutual TLS)	Client presents certificate; server validates against CA. Both parties authenticate.	Very strong, no passwords, phishing-resistant	Complex cert management, not browser-friendly	Service-to-service in zero-trust, banking APIs
SAML 2.0	Enterprise SSO. IdP sends signed XML assertion to SP after user authenticates.	Enterprise standard, SSO across orgs	XML complexity, verbose, hard to debug	Enterprise SSO, government
Passkey (FIDO2/WebAuthn)	Cryptographic key pair on device. Private key never leaves device. Biometric or PIN unlocks it.	Phishing-resistant, passwordless, UX good	Device dependency, recovery complexity	Modern consumer apps, next-gen auth

JWT Deep Dive

JWT Structure

```
// Header.Payload.Signature (Base64URL encoded) // Header: {"alg":"HS256","typ":"JWT"} // Payload: {"sub":"user123","tenantId":"abc","roles":["ADMIN"],"iat":1718000000,"exp":1718003600} // Signature: HMACSHA256(base64url(header)+"."+base64url(payload), secret) // Spring Security JWT validation @Bean public JwtDecoder jwtDecoder() { return NimbusJwtDecoder.withSecretKey( new SecretKeySpec(secret.getBytes(), " HmacSHA256")).build(); // For RS256 (asymmetric, preferred for microservices): // return NimbusJwtDecoder.withPublicKey(rsaPublicKey).build(); } // Access token: short-lived (15 min). Refresh token: long-lived (7-30 days), stored securely.
```

Claim	Meaning	Best Practice
sub	Subject — user ID	Use opaque ID, not email
exp	Expiry timestamp (Unix)	Short TTL: 15 min for access token
iat	Issued at timestamp	Include always
jti	JWT ID — unique per token	Include for revocation capability
roles/scopes	Authorization claims	Keep small — avoid large payloads
iss	Issuer URL	Validate issuer on receipt
aud	Audience — intended recipient	Validate audience on receipt

OAuth2 Flows

Flow	When	Steps
Authorization Code + PKCE	Web/mobile app, user-facing. Most secure flow.	1. Redirect to IdP with code_challenge 2. User authenticates 3. IdP returns code 4. App exchanges code+verifier for tokens 5. App uses access token
Client Credentials	Machine-to-machine (no user). Service A calls Service B.	1. Service sends client_id + client_secret to IdP 2. IdP returns access token 3. Service uses token to call API
Refresh Token	Silently renew expired access token.	1. Access token expires 2. Client sends refresh token to IdP 3. IdP returns new access token (+ optionally new refresh token)
Device Code	Smart TV, CLI tools — no browser redirect.	1. Device gets device_code + user_code 2. User visits URL on another device, enters user_code 3. Device polls until token granted

Authorization Strategies

1. RBAC — Role-Based Access Control

```
// Users have roles; roles have permissions // User: ADMIN, MANAGER, VIEWER @PreAuthorize("hasRole('ADMIN') or hasRole('MANAGER')") public Order updateOrder(Long id, OrderDTO dto) { ... } // Spring Security config . requestMatchers("/admin/**").hasRole("ADMIN") .requestMatchers("/orders").hasAnyRole("ADMIN","MANAGER","VIEWER")
```

2. ABAC — Attribute-Based Access Control

```
// Policy: user can edit order IF user.department == order.department AND order.status != CLOSED @PreAuthorize("@authzService.canEditOrder(authentication, #orderId)") public Order editOrder(Long orderId, OrderDTO dto) { ... } @Service public class AuthzService { public boolean canEditOrder(Authentication auth, Long orderId) { UserDetails user = (UserDetails) auth.getPrincipal(); Order order = orderRepo.findById(orderId).orElseThrow(); return user.getDepartment().equals(order.getDepartment()) && order.getStatus() != OrderStatus.CLOSED; } }
```

3. ReBAC — Relationship-Based Access Control (Google Zanzibar model)

Authorization based on relationships in a graph. "User A can edit Document D if A is an owner of D, or A is a member of a group that is an owner of D." Used by Google Drive, GitHub, Notion. Implemented via policy engines like OpenFGA, Ory Keto, SpiceDB.

4. ACL — Access Control List

```
// Per-resource permissions list // Document 123: [user:alice -> READ, WRITE], [user:bob -> READ] PERMIT user:alice ON resource:document:123 ACTIONS:read,write PERMIT user:bob ON resource:document:123 ACTIONS:read
```

Model	Best For	Scalability	Flexibility
RBAC	Enterprise apps with clear role hierarchy	High	Low — role explosion with many permissions
ABAC	Fine-grained dynamic policies (time, location, dept)	Medium	Very high
ReBAC	Resource ownership graphs, collaborative apps	High (Zanzibar)	High
ACL	Per-resource permissions, small user sets	Low at scale	Medium

Spring Security Configuration

```
@Configuration @EnableWebSecurity @EnableMethodSecurity public class SecurityConfig { @Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { return http . csrf(AbstractHttpConfigurer::disable) // stateless API .sessionManagement(s -> s.sessionCreationPolicy(STATELESS)) .authorizeHttpRequests(a -> a .requestMatchers("/api/auth/**").permitAll() . requestMatchers("/api/admin/**").hasRole("ADMIN") .requestMatchers(HttpMethod.GET, " /api/products/**").hasAnyRole("USER","ADMIN") .anyRequest().authenticated() ) .oauth2ResourceServer(o -> o.jwt(j -> j.decoder(jwtDecoder()))) .addFilterBefore(new TenantFilter(), UsernamePasswordAuthenticationFilter.class) .build(); } // Extract roles from JWT claims @Bean public JwtAuthenticationConverter jwtAuthConverter() { JwtGrantedAuthoritiesConverter conv = new JwtGrantedAuthoritiesConverter(); conv.setAuthoritiesClaimName("roles"); // custom claim name conv.setAuthorityPrefix("ROLE_"); // Spring convention JwtAuthenticationConverter jac = new JwtAuthenticationConverter(); jac.setJwtGrantedAuthoritiesConverter(conv); return jac; } }
```

Token Storage & Security Best Practices

Practice	Detail
Store access token in memory (SPA)	Never in localStorage — XSS can steal it. Store in JS memory variable. Loses on page refresh — use refresh token flow to restore.
Store refresh token in HttpOnly cookie	HttpOnly: cannot be read by JavaScript. Secure: HTTPS only. SameSite=Strict or Lax for CSRF protection.
Short access token TTL	15 minutes. Limits blast radius if token is stolen.
Rotate refresh tokens	Issue new refresh token on each refresh. Invalidate old one. Detect token reuse as compromise signal.
Validate all claims	Verify: signature, exp (not expired), iss (matches your IdP), aud (matches your API). Never skip any.
Use RS256 not HS256	Asymmetric RS256: public key can validate tokens. Private key only on IdP. HS256 requires sharing secret with every service.
Rate limit login endpoint	Prevent brute force. Lockout after N failures. Use CAPTCHA or progressive delays.
Log auth events	Log: login success/failure, token issue, role change, permission denied. Include IP, user-agent, timestamp. Alert on anomalies.

Interview Q&A;

Q1: What is the difference between 401 and 403?

401 Unauthorized means the request lacks valid authentication credentials — the user is not logged in or the token is missing/expired. 403 Forbidden means the user is authenticated but does not have permission to access the resource. Common mistake: returning 401 when a logged-in user tries to access a resource they're not allowed — that should be 403.

Q2: Why is JWT stateless and what is its main security risk?

JWT is stateless because the server doesn't store sessions — all claims are in the token itself. Signature verification is enough. The main risk is inability to immediately revoke a token before its expiry. If a JWT is stolen, it's valid until exp. Mitigations: short TTL (15 min), maintain a server-side blocklist of revoked jti values checked on each request, or use short-lived opaque tokens with introspection.

Q3: What is PKCE and why is it needed?

PKCE (Proof Key for Code Exchange) prevents authorization code interception attacks in OAuth2. The client generates a random code_verifier, hashes it to code_challenge, and sends the challenge to the authorization server. When exchanging the code for a token, the client sends the original verifier — the server hashes it and compares. An intercepted code is useless without the verifier. Required for all public clients (SPAs, mobile apps) that can't keep a client_secret safe.

Q4: What is ABAC and when would you prefer it over RBAC?

ABAC (Attribute-Based Access Control) makes authorization decisions based on attributes of the user, resource, and environment. Example policy: "Managers can approve expenses IF the amount is under $10,000 AND the expense is in their department AND it's during business hours." RBAC is simpler and faster, but roles multiply when you need fine-grained rules. Choose ABAC when permissions depend on contextual data that changes at runtime.

Q5: How would you implement API key authentication in Spring Boot?

Create an OncePerRequestFilter that: (1) extracts the X-API-Key header; (2) looks up the key in Redis or DB (cache it!); (3) if valid, creates a PreAuthenticatedAuthenticationToken with the key's associated user/scopes; (4) sets it in SecurityContextHolder. The key is stored hashed (SHA-256) in DB — never plaintext. Include: key ID (prefix for identification), created_at, last_used_at, expiry, rate_limit tier, enabled flag.

Q6: What is the difference between OAuth2 Authorization Code flow and Client Credentials flow?

Authorization Code + PKCE is for user-facing flows — the user authenticates with the IdP and grants an app access to their data. It involves browser redirects, user consent. Client Credentials is for machine-to-machine — no user involved. A service authenticates directly with the IdP using its client_id + client_secret to get an access token to call another service. No user consent, no browser.

Q7: How do you secure inter-service communication in microservices?

(1) JWT propagation: forward the user's JWT to downstream services; validate it at each service. (2) Service-to-service: Client Credentials OAuth2 flow — each service has its own client_id/secret to get service tokens. (3) mTLS: each service has a certificate; mutual TLS authentication between services. (4) API Gateway: gateway validates all inbound tokens; internal services trust gateway-signed headers. Use a service mesh (Istio, Linkerd) for mTLS at infrastructure level.

Q8: What are common authorization vulnerabilities?

(1) IDOR (Insecure Direct Object Reference): user accesses /api/orders/456 without server checking if order 456 belongs to them; (2) Missing function-level access control: endpoint accessible without role check; (3) Privilege escalation: user changes their own role via an unsecured API; (4) JWT algorithm confusion (alg:none attack): server accepts unsigned JWT if alg validation is missing; (5) Over-broad scopes: issuing admin-scoped tokens when only read is needed. Fix: always validate resource ownership, use @PreAuthorize, verify token algorithm explicitly.